

UNIVERSIDAD AUTONOMA DEL BENI “JOSE BALLIVIÁN”
FACULTAD DE INGENIARIA Y TECNOLOGIA
CARRERA DE INGENIAERIA DE SISTEMAS

DOCENTE: Hermes Rodrigesz Rivero

MATERIA: TECNOLOGIAS EMERGENTES

SEMESTRE: 7mo

UNIV.: Aruro Camader Guamayo

Betxy Ovale Vasquez

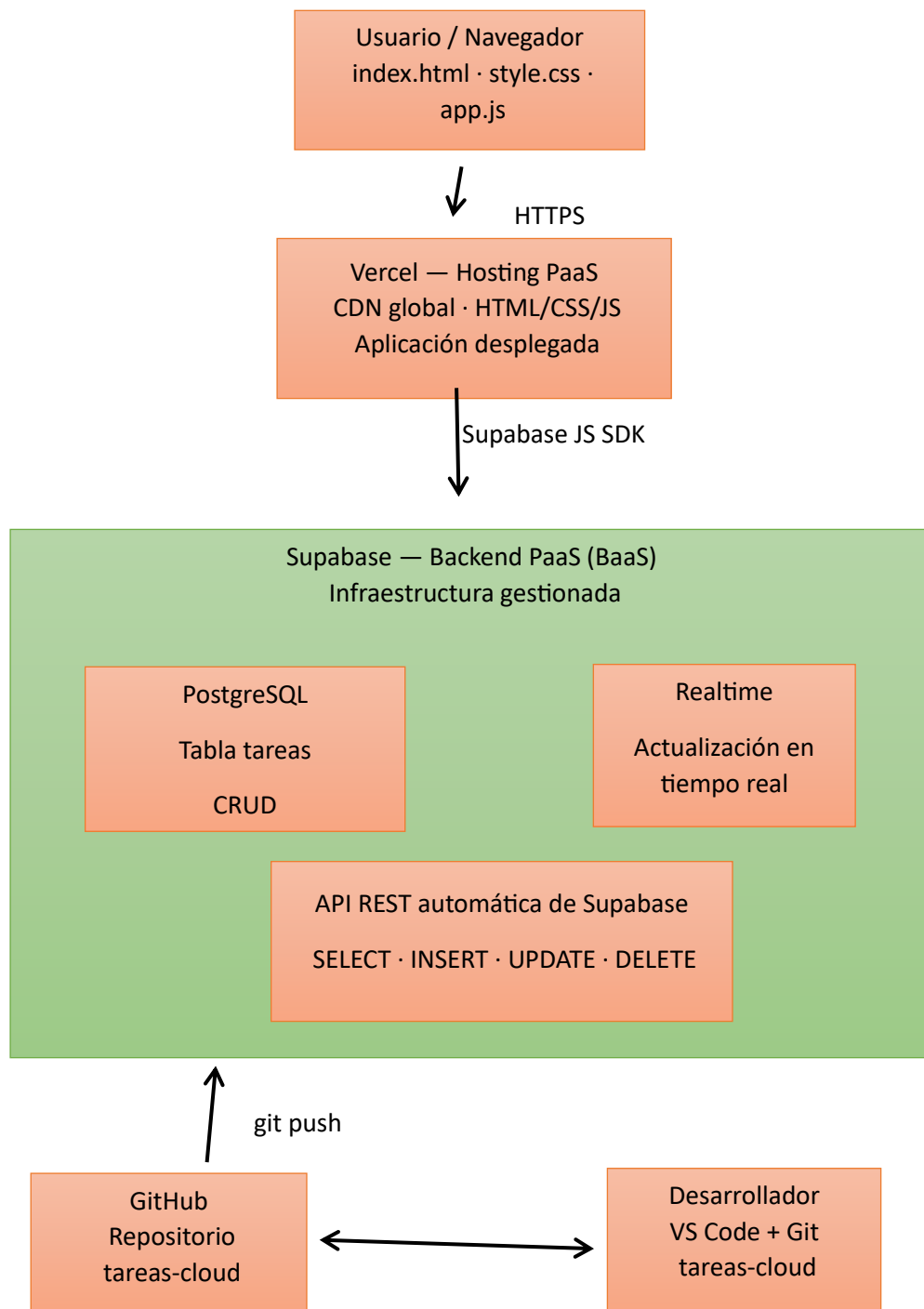
Kevin Romero Vargas

INFORME TÉCNICO

1. Introducción

La presente práctica tuvo como finalidad desarrollar una aplicación web colaborativa utilizando herramientas basadas en Cloud Computing. Se implementó un sistema de registro de tareas en tiempo real empleando Supabase como backend gestionado, Git y GitHub para control de versiones y Vercel para el despliegue.

Descripción de la arquitectura cloud implementada



Clasificación del Modelo de Servicio Cloud, Ventajas y Limitaciones

Clasificación del Modelo de Servicio Cloud

Durante el desarrollo del proyecto **Tareas Cloud** se utilizaron diversas herramientas relacionadas con la computación en la nube. Cada una cumple una función específica dentro de la arquitectura implementada.

Supabase se clasifica como **PaaS (Platform as a Service)** porque proporciona una plataforma completa para el desarrollo de aplicaciones, incluyendo base de datos PostgreSQL, API REST y sincronización en tiempo real. Gracias a este servicio, fue posible almacenar y gestionar la información de las tareas sin necesidad de administrar servidores o infraestructura propia.

Vercel también corresponde al modelo **PaaS**, ya que permite desplegar aplicaciones web de forma rápida y sencilla. La plataforma se encarga del alojamiento, distribución y disponibilidad del sistema en Internet, permitiendo que los desarrolladores se concentren únicamente en el desarrollo de la aplicación.

Por otra parte, **GitHub** pertenece al modelo **SaaS (Software as a Service)**. Este servicio fue utilizado para almacenar el código fuente del proyecto, registrar cambios mediante commits y facilitar la integración con Vercel para el despliegue automático de la aplicación.

Finalmente, herramientas como **Git** y **Visual Studio Code** no se clasifican dentro de los modelos IaaS, PaaS o SaaS, ya que son aplicaciones instaladas localmente que sirven para el desarrollo y control de versiones del proyecto.

Ventajas Encontradas

El uso de tecnologías cloud permitió desarrollar la aplicación de manera más rápida y eficiente. Una de las principales ventajas fue la facilidad de integración entre Supabase, GitHub y Vercel, lo que simplificó el proceso de desarrollo y despliegue.

Otra ventaja importante fue la actualización de datos en tiempo real proporcionada por Supabase, permitiendo que los cambios realizados en las tareas se reflejen inmediatamente en la aplicación. Asimismo, Vercel facilitó la publicación del proyecto en Internet sin necesidad de realizar configuraciones complejas de servidores.

GitHub aportó seguridad y organización mediante el control de versiones, permitiendo registrar cada modificación realizada durante el desarrollo. Además, el uso de servicios gratuitos ayudó a reducir costos y demostró cómo las soluciones cloud permiten crear aplicaciones funcionales sin invertir en infraestructura propia.

Limitaciones Encontradas

Durante el desarrollo también se identificaron algunas limitaciones. La principal fue la necesidad de aprender el funcionamiento de Supabase y su proceso de configuración, especialmente en la conexión con la aplicación mediante claves y APIs.

Asimismo, el proyecto depende completamente de una conexión a Internet para acceder a los servicios cloud y administrar la aplicación. Otra limitación corresponde a las restricciones de los planes gratuitos de las plataformas utilizadas, que pueden resultar insuficientes para proyectos de gran tamaño.

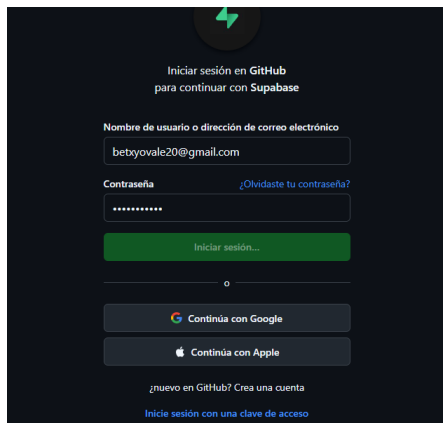
Finalmente, la dependencia de servicios externos implica que cualquier problema técnico o interrupción en las plataformas utilizadas podría afectar temporalmente el funcionamiento del sistema.

En general, las ventajas obtenidas fueron mayores que las limitaciones encontradas, permitiendo desarrollar una aplicación web funcional y adquirir experiencia práctica en el uso de tecnologías de computación en la nube.

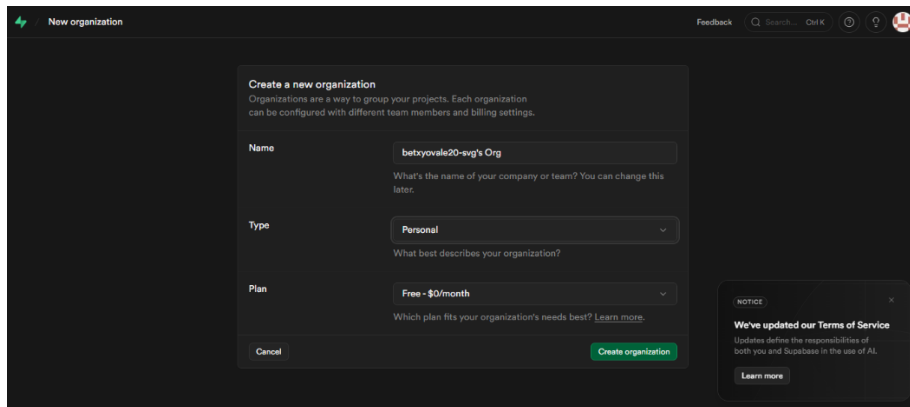
Capturas de pantalla de cada fase

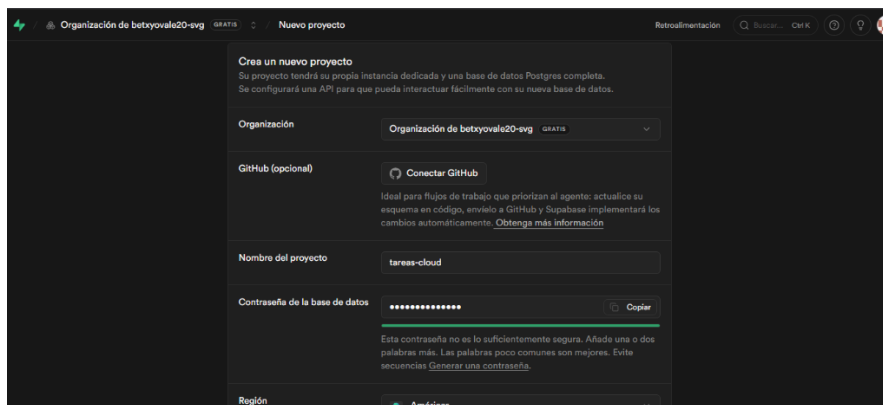
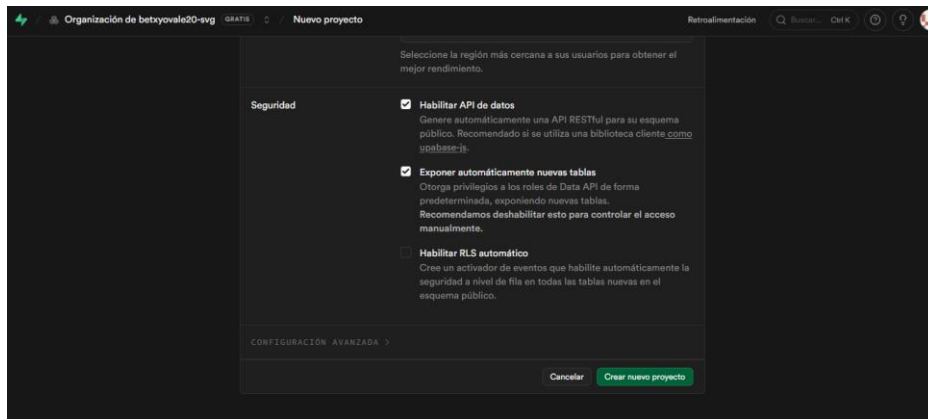
FASE 1: Configuración de Supabase (PaaS – Backend en la Nube)

1. Ingresar a <https://supabase.com> y crear una cuenta gratuita (puede usarse la cuenta de GitHub).

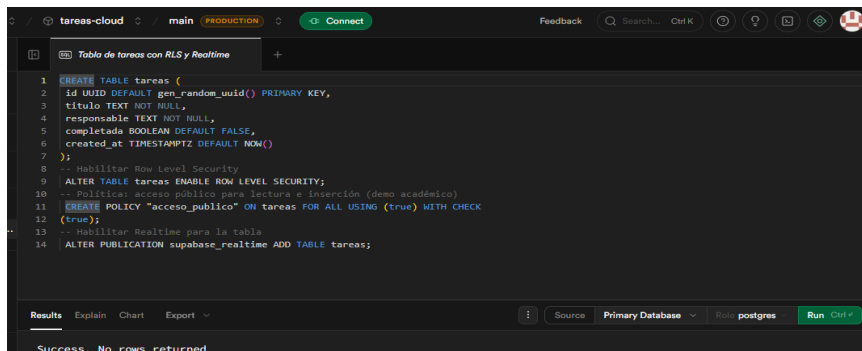


2. Crear un nuevo proyecto. Elegir un nombre, contraseña segura para la base de datos y la región más cercana (por ejemplo: South America - São Paulo).

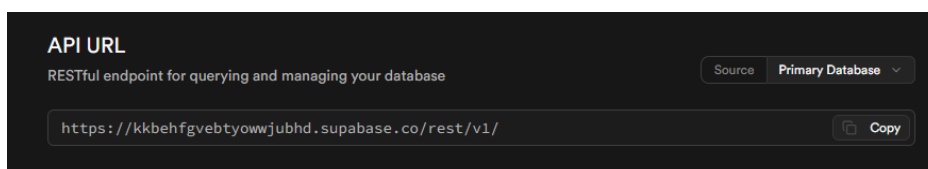


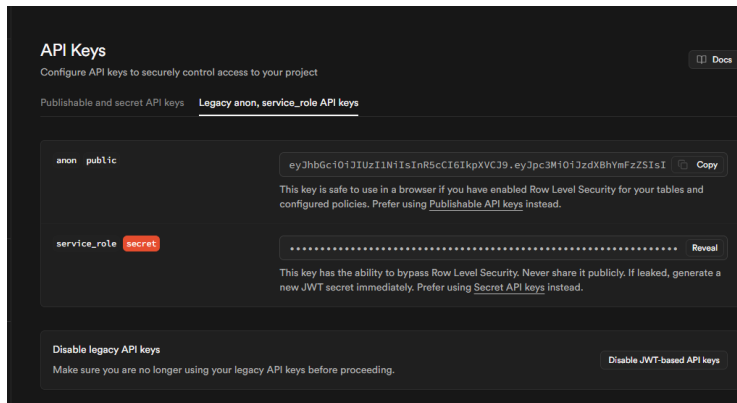


3. Una vez creado el proyecto, ir a SQL Editor y ejecutar el siguiente script para crear la tabla:



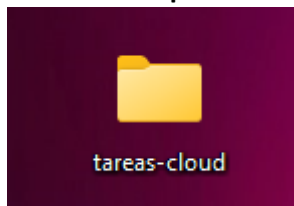
4. En la sección Project Settings > API, copiar los valores de: Project URL y anon public key. Estos serán las credenciales de la aplicación.



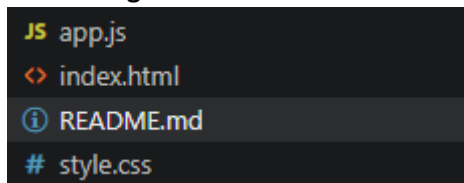


FASE 2: Estructura del Proyecto en Visual Studio Code

1. Crear una carpeta local llamada tareas-cloud.

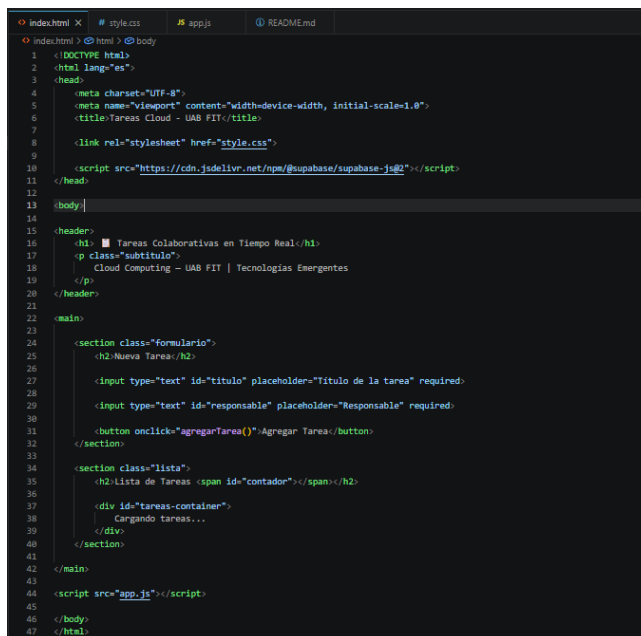


2. Abrir la carpeta con VS Code (File > Open Folder)
3. Crear la siguiente estructura de archivos:



FASE 3: Desarrollo de la Aplicación

1. Crear el archivo index.html



2. Crea el archivo app.js con la lógica conexión a Supabase

```

1 // IMPORTANTE: Reemplazar con tus credenciales de Supabase
2 const SUPABASE_URL = 'https://kkbehfgvbytyoww7ubhd.supabase.co';
3 const SUPABASE_ANON_KEY = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ3c3MiOiJzdXBhYmFzZSI6IjEwIHRlbnRpdjkiOiJkIiwiaXNja3kiOiJkIiwiaWF0Ijoi1610101010IiwiaWF0Ijoi1610101010InQ';
4
5 const { createClient } = supabase;
6 const db = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
7
8 // Cargar tareas al iniciar
9 async function cargarTareas() {
10   const { data, error } = await db
11     .from('tareas')
12     .select('*')
13     .order('created_at');
14
15   if (error) {
16     console.error(error);
17     return;
18   }
19
20   renderTareas(data);
21 }
22
23 // Suscripción en tiempo real
24 db.channel('tareas-canal')
25   .on(
26     'postgres_changes',
27     {
28       event: '*',
29       schema: 'public',
30       table: 'tareas'
31     },
32     () => {

```

3. Crea el archivo style.css

```

1 /*
2   *{
3     margin:0;
4     padding:0;
5     box-sizing:border-box;
6   }
7
8   body{
9     font-family:'Segoe UI',sans-serif;
10    background:linear-gradient(135deg,#1f0f2a,#1e293b);
11    min-height:100vh;
12    color:#462fad;
13  }
14
15  header{
16    text-align:center;
17    color:white;
18    padding:35px;
19  }
20
21  header h1{
22    font-size:2.2rem;
23    margin-bottom:10px;
24  }
25
26  .subtitulo{
27    color:#b42947;
28  }
29
30  main{
31    max-width:1000px;
32    margin:auto;
33    padding:20px;
34  }
35
36  .formulario,
37  .lista{
38    background:white;
39    border-radius:20px;
40    padding:25px;

```

4. Crea el archivo README.md

```

① README.md
1  # Tareas Cloud
2
3  ## Descripción
4
5  Tareas Cloud es una aplicación web colaborativa desarrollada para gestionar tareas en tiempo real utilizando
   tecnologías cloud. La aplicación permite registrar tareas, asignar responsables, marcar tareas como completadas
   y eliminarlas.
6
7  ## Arquitectura
8
9  La aplicación está compuesta por:
10
11  * Frontend: HTML, CSS y JavaScript.
12  * Backend como servicio (BaaS): Supabase.
13  * Control de versiones: Git y GitHub.
14  * Despliegue en la nube: Vercel.
15
16  ### Flujo de funcionamiento
17
18  Usuario → Aplicación Web → Supabase → Actualización en tiempo real → Usuario
19
20  ## Tecnologías utilizadas
21
22  * HTML5
23  * CSS3
24  * JavaScript
25  * Supabase
26  * Git
27  * GitHub
28  * Vercel
29
30  ## Capturas de pantalla
31
32  ### Pantalla principal
33
34  ![Aplicación](capturas/aplicacion.png)
35
36  ### Base de datos en Supabase
37

```

FASE 4: Control de versiones con git y github

1. Inicializar el repositorio git en la terminal integrada de VS code

```

PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud> git init
Initialized empty Git repository in C:/Users/Betxy/OneDrive/Desktop/tareas-cloud/.git/
PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud>

```

```

PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud> git add .
PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud> git commit -m "feat: primer commit - aplicación tareas cloud"

```

```

Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'Betxy@LAPTOP-8BAC4PH4.(none)')
PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud>

```

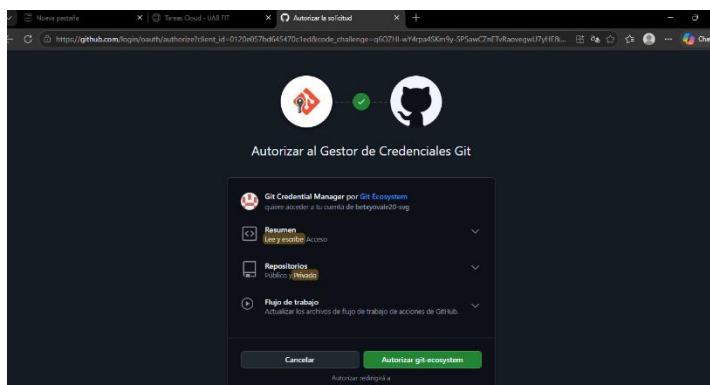
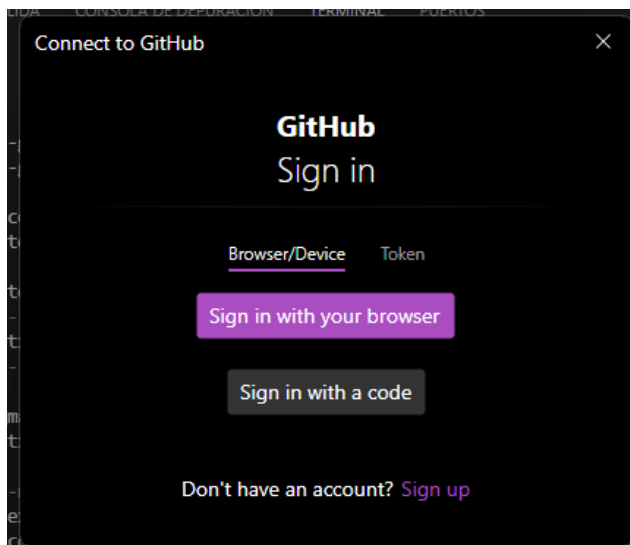

1. Crea un nuevo repositorio en github

```
PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud> git init
>> git add .
>> git commit -m "feat: primer commit - aplicación tareas cloud"
Reinitialized existing Git repository in C:/Users/Betxy/OneDrive/Desktop/tareas-cloud/.git/
[master (root-commit) cc3603d] feat: primer commit - aplicación tareas cloud
3 files changed, 346 insertions(+)
create mode 100644 app.js
create mode 100644 index.html
create mode 100644 style.css
PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud>
```

```
fatal: unable to auto-detect email address (got 'betxy@betxyovale20-sv.com')
>> git config --global user.name "betxy"
>> git config --global user.email "betxyovale20@gmail.com"
PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud> git config --global user.name
betxy
betxyovale20@gmail.com
PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud>
```

2. Vincular el repositorio local con github y subir el código

```
PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud> git remote add origin https://github.com/betxyovale20-svg/tareas-cloud.git
>> git branch -M main
>> git push -u origin main
```



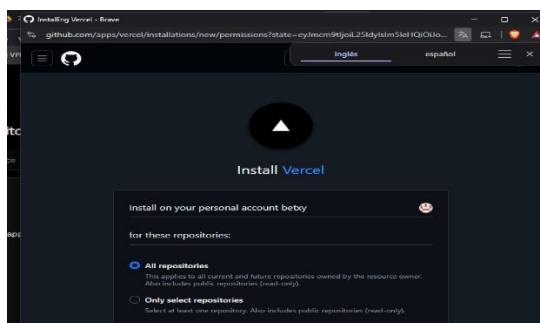
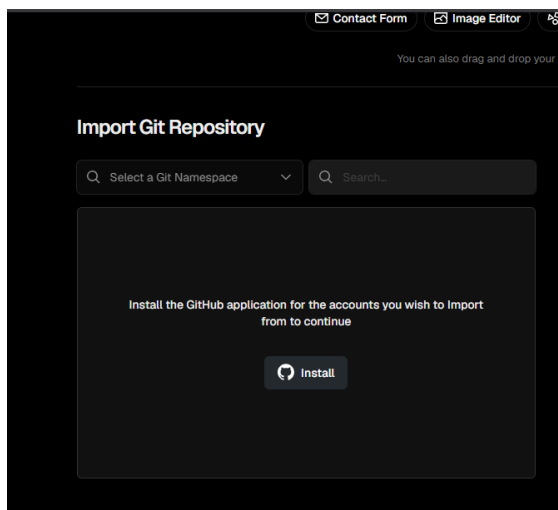
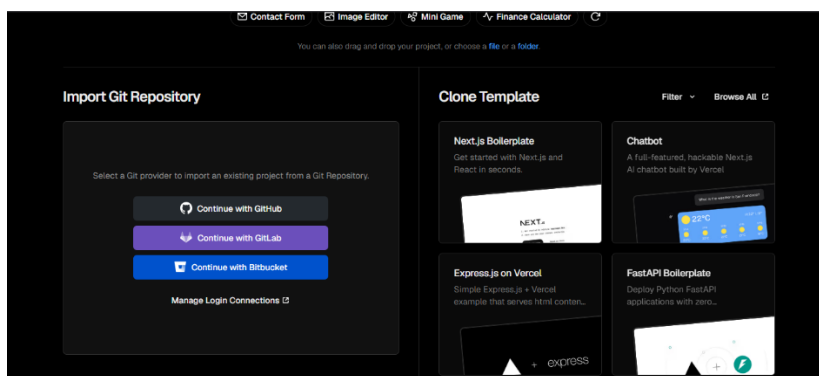
```

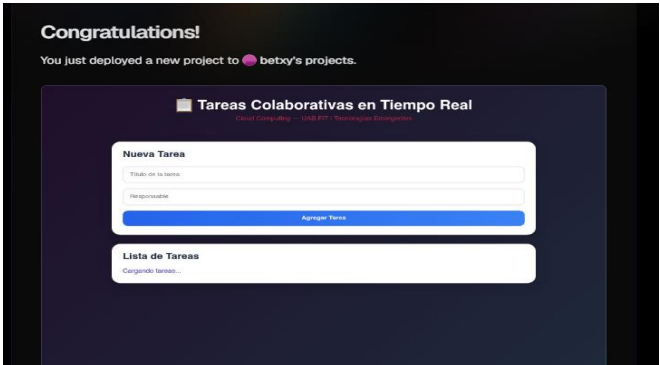
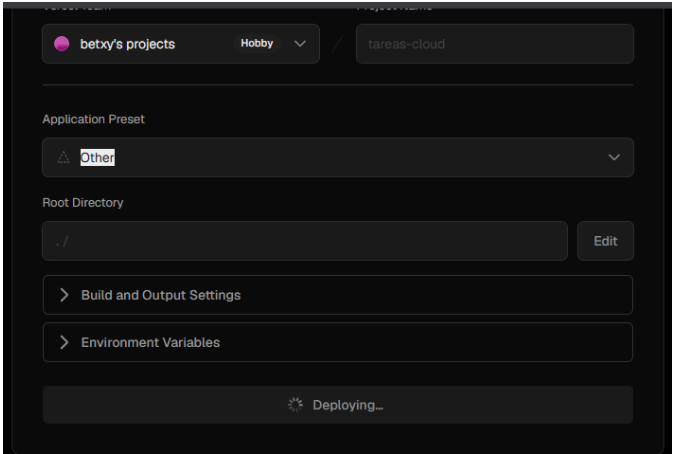
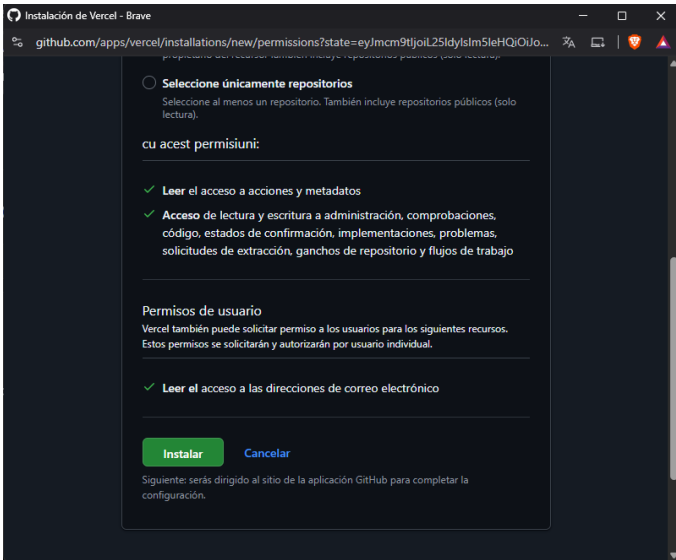
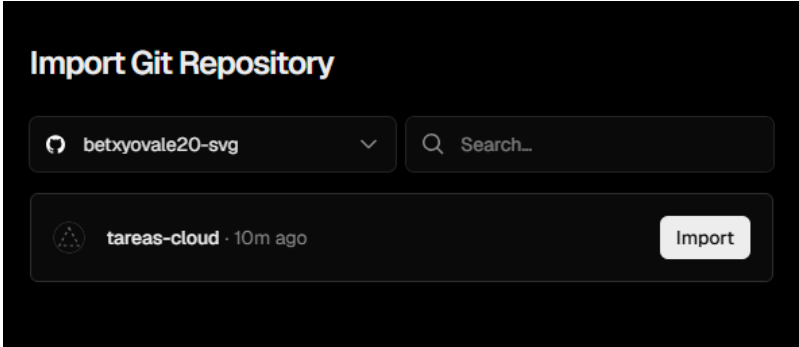
PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud> git remote add origin https://github.com/betxyovale20-svg/tareas-cl
oud.git
>> git branch -M main
>> git push -u origin main
error: remote origin already exists.
info: please complete authentication in your browser...
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 2.90 KiB | 156.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/betxyovale20-svg/tareas-cloud.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Betxy\OneDrive\Desktop\tareas-cloud>

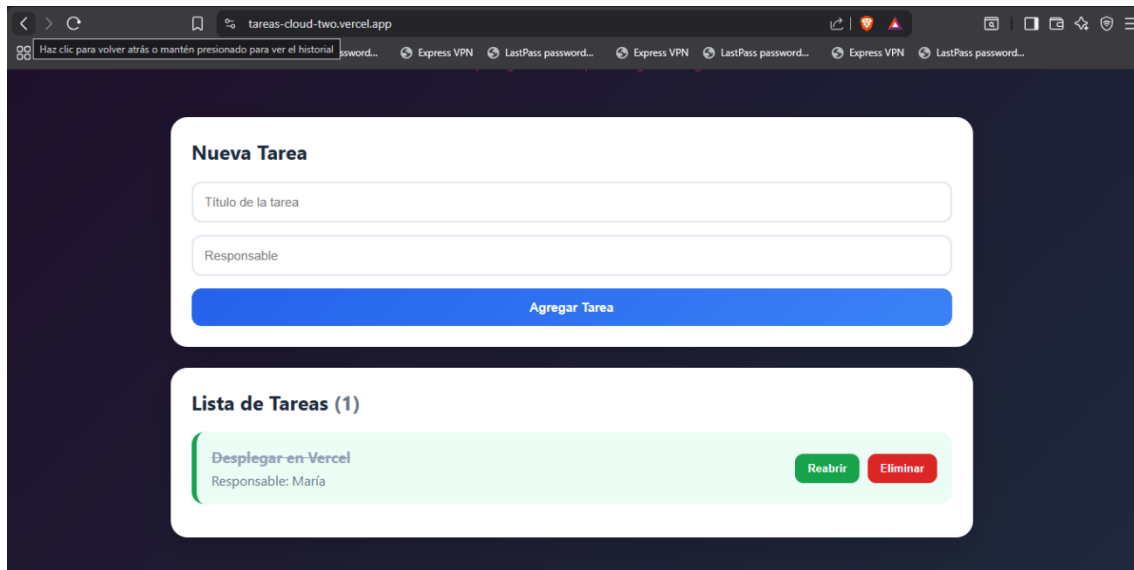
```

FASE 5: Despliegue en vercel (PaaS de Hoating)

1. Ingrese a <https://vercel.com> y crea una cuenta







REFLECCIÓN

1. ¿En qué modelo de servicio clasificarías a Supabase y por qué?

Supabase se clasifica como PaaS porque proporciona una plataforma completa para desarrollar aplicaciones, incluyendo base de datos, API, autenticación y servicios en tiempo real, sin necesidad de administrar servidores.

2. ¿Qué tareas del backend manejó Supabase?

Creación y gestión de la base de datos.

API REST automática.

Realtime.

Seguridad mediante RLS.

Almacenamiento de datos.

3. ¿Cómo funciona el tiempo real en Supabase?

Supabase utiliza WebSockets para mantener una conexión permanente entre cliente y servidor. Cuando ocurre un cambio en la base de datos, este se notifica inmediatamente a todos los clientes conectados.

4. ¿Cuál fue el proceso de CI/CD implementado?

Desarrollo local.

Commit con Git.

Push a GitHub.

GitHub almacena el código.

Vercel detecta cambios.

Despliegue automático.

Publicación en producción.

5. ¿Qué ventajas y desventajas encontraste en el modelo PaaS?

Ventajas

- Rapidez.
- Menor costo.
- Escalabilidad.
- Mantenimiento simplificado.

Desventajas

- Dependencia del proveedor.
- Menor control.
- Limitaciones del plan gratuito.

6. ¿Qué consideraciones de seguridad aplicarías en producción real?

- Autenticación de usuarios.
- Roles y permisos.
- Políticas RLS estrictas.
- HTTPS obligatorio.
- Protección de claves API.
- Validación de datos.
- Copias de seguridad.
- Monitoreo continuo.